

JavaScript Specials – Simple Hinglish Summary

Source: <https://javascript.info/javascript-specials>

Main baat

Ye chapter ab tak seekhi hui JavaScript ki cheezon ka **short recap** hai, khaaskar un baaton par jahan galti hone ke chances zyada hain.

1. Code structure

- Statements **semicolon** (;) se alag hote hain:

```
javascript  
  
alert('Hello'); alert('World');
```

- Aam taur par **line break** bhi separator maana jata hai (**automatic semicolon insertion**):

```
javascript  
  
alert('Hello')  
alert('World')
```

- Ye kabhi kabhi fail hota hai. Jaise:

```
javascript  
  
alert("There will be an error after this message")  
  
[1, 2].forEach(alert)
```

- Zyadatar style guides** kehte hain: har statement ke baad semicolon lagao.
- Code blocks** {...} ke baad semicolon zaruri nahi (function declaration, loops). Extra semicolon laga do to error nahi aata, ignore ho jata hai.

2. Strict mode

- Modern JavaScript ke saare features ON karne ke liye script ke **sabse upar** "use strict" likho:

```
javascript  
  
'use strict';
```

- Ye script ke top par ya function body ke start me hona chahiye.
- Iske bina bhi sab chalta hai, lekin kuch features **purane "compatible" tarike** se behave karte hain.
- Kuch modern features (jaise **classes**) strict mode **automatically** ON kar dete hain.

3. Variables

Declare karne ke tarike:

- `let`
- `const` (constant, badal nahi sakta)
- `var` (purana tarika)

Naam me kya ho sakta hai:

- Letters aur digits, lekin **pehla character digit nahi** ho sakta.
- `$` aur `_` normal characters hain, letters ke barabar.
- Non-Latin alphabets bhi allowed hain, par aam taur par use nahi hote.

Dynamically typed: variable me kuch bhi rakh sakte ho:

```
javascript
let x = 5;
x = "John";
```

8 data types:

- `number` - integer aur decimal
- `bigint` - kisi bhi length ke integers
- `string` - text
- `boolean` - `true/false`
- `null` - "empty" ya "exist nahi karta"
- `undefined` - "assign nahi hua"
- `object` aur `symbol` - complex data structures aur unique identifiers (abhi nahi seekhe)

`typeof` operator type batata hai, do exceptions ke saath:

```
javascript
typeof null == "object"           // language ki galti
typeof function(){} == "function" // functions special treat hote hain
```

4. Interaction (Browser ke functions)

- `prompt(question, [default])`: sawal poochta hai. User ne jo likha wo return karta hai, ya Cancel par `null`.
- `confirm(question)`: OK/Cancel poochta hai. `true/false` return karta hai.
- `alert(message)`: message dikhata hai.

Teeno **modal** hain: code execution pause karte hain aur user ko page se interact nahi karne dete jab tak jawab na de.

javascript

```
let userName = prompt("Your name?", "Alice");
let isTeaWanted = confirm("Do you want some tea?");

alert( "Visitor: " + userName );           // Alice
alert( "Tea wanted: " + isTeaWanted );    // true
```

5. Operators

Arithmetical: `*` `+` `-` `/`, `%` (remainder), `**` (power). Binary `+` strings ko jodta hai. Koi ek operand string ho to dusra bhi string ban jata hai:

javascript

```
alert( '1' + 2 ); // '12'
alert( 1 + '2' ); // '12'
```

Assignments: simple `a = b` aur combined jaise `a *= 2`.

Bitwise: 32-bit integers par bit-level par kaam karte hain. Zarurat padne par docs dekho.

Conditional (ternary): akela operator jisme 3 parts hain: `cond ? resultA : resultB`. `cond` truthy ho to `resultA`, warna `resultB`.

Logical operators:

- `&&` aur `||` **short-circuit** karte hain aur us jagah ki **original value** return karte hain jahan ruke (zaruri nahi `true/false`).
- `!` operand ko boolean me convert karke **ulta** return karta hai.

Nullish coalescing `??`: defined value chunta hai. `a ?? b` ka result `a` hai, jab tak wo `null/undefined` na ho, tab `b`.

Comparisons:

- `==` alag types ko **number me convert** karta hai (sirf `null` aur `undefined` iske exception hain, wo sirf ek dusre ke barabar hain). Isliye ye sab `true` hain:

javascript

```
alert( 0 == false ); // true
alert( 0 == '' );    // true
```

- Baaki comparisons bhi number me convert karte hain.
- `===` **strict equality**: conversion nahi karta. Alag types = alag values.
- Greater/less comparisons **strings ko character-by-character** compare karte hain, baaki types number me convert hote hain.

Aur operators: comma operator jaise kuch aur bhi hain.

6. Loops

Teen type ke loops:

javascript

```
// 1
while (condition) {
    ...
}

// 2
do {
    ...
} while (condition);

// 3
for (let i = 0; i < 10; i++) {
    ...
}
```

- `for(let...)` me declare kiya variable **sirf loop ke andar** dikhta hai. `let` hata kar pehle se bana variable bhi use kar sakte ho.
- `break` poore loop se nikalta hai, `continue` current iteration chhodta hai. **Nested loops** todne ke liye **labels** use karo.

7. `switch`

Kai `if` checks ki jagah use hota hai. Comparison **strict** (`===`) hota hai.

javascript

```
let age = prompt('Your age?', 18);

switch (age) {
  case 18:
    alert("Won't work"); // prompt string deta hai, number nahi
    break;

  case "18":
    alert("This works!");
    break;

  default:
    alert("Any value not equal to one above");
}
```

8. Functions

Function banane ke **teen tarike**:

1. Function Declaration (main code flow me):

javascript

```
function sum(a, b) {
  let result = a + b;

  return result;
}
```

2. Function Expression (expression ke andar):

javascript

```
let sum = function(a, b) {
  let result = a + b;

  return result;
};
```

3. Arrow functions:

javascript

```
// right side me expression
let sum = (a, b) => a + b;

// ya multi-line, yahan return zaruri
let sum = (a, b) => {
  // ...
  return a + b;
}

// bina arguments ke
let sayHi = () => alert("Hello");

// ek argument ke saath
let double = n => n * 2;
```

Dhyan rakhne wali baatein:

- Function ke **local variables** wo hote hain jo body me ya parameter list me declare hon. Ye sirf function ke andar dikhte hain.
- Parameters ki **default values** ho sakti hain: `function sum(a = 1, b = 2) {...}`
- Function **hamesha kuch return karta hai**. `return` na ho to `undefined`.

Quick Recap Table

Topic	Yaad rakhne wali baat
Semicolon	Har statement ke baad lagao
Strict mode	Script ke top par <code>"use strict"</code>
Variables	<code>let</code> , <code>const</code> use karo, <code>var</code> nahi
Data types	8 hain (7 primitive + <code>object</code>)
<code>typeof null</code>	<code>"object"</code> (language ki galti)
<code>+</code>	String ho to jodta hai
<code>==</code> vs <code>===</code>	<code>===</code> use karo (conversion nahi)
<code> </code> , <code>&&</code>	Original value return karte hain
<code>??</code>	Sirf <code>null/undefined</code> par default deta hai
<code>switch</code>	Strict comparison
Functions	Declaration, Expression, Arrow
Return na ho	<code>undefined</code> milta hai